SOE104: Java Programming — Coursework 2, Semester 1, 2026-27

Programming Project — Oregon Trail

Read carefully — no dispensation will be given for lack of awareness of the rules.

Assignment type	Group programming assignment in teams of 4–5 (allocations on Learning Mall). Your team shares one platform account (a team number) and submits together.
Weighting	100 marks, 35% of the module. All marking is automatic (§5).
Release date	Thursday, 5 November 2026
Due date	Sunday, 13 December 2026, 23:59 (China time)
Submission platform	The course auto-grading platform (URL on LM). Log in with your team number and initial password (first login forces a password change). Upload your Java sources — the platform compiles, runs and scores them automatically, and shows a per-turn replay of your journey.
Resubmission	Unlimited before the deadline. Only your latest submission counts; full history is kept.
Late submissions	Handled automatically: 5% per day after the due date, capped at 25% (exact rate on the assignment page).
Special consideration	Only via formal MC application through e-Bridge.
Artificial Intelligence	You must not use AI to produce this coursework for you. What is acceptable: discussing the rules and Java concepts with AI assistants, asking for explanations, hints or feedback — and then writing and understanding every line of code yourselves. What is not acceptable: having an AI generate your submission, in whole or in substantial part. The platform runs automated screening (submission patterns and code similarity) that flags likely AI-completed work; flagged teams will be invited to a short **design defense** in which you must explain and justify your party's design decisions. Please cite any AI assistance you used (tool and purpose) in a code-comment header. An inability to explain your own submission is treated under the University's Academic Integrity Policy.
Plagiarism	The platform runs automatic code-similarity detection across all teams (current and historical). Copying between teams is referred under the University's Academic Integrity Policy.
Peer evaluation	Each member submits a peer evaluation form on LM; individual marks may be adjusted for unequal contribution.
Tips	Read §4 (sandbox rules — some violations score zero). Submit early and iterate: the replay is the best debugging tool you have.

The year is 1848. Your team's wagon party is heading west along the Oregon Trail. You do not steer the wagon yourself — your Java code does. You design a party of travellers (their roles, their behaviour, the party's strategy, the outfitting of the wagon) as an object-oriented system. The instructors' engine sends every team's party down the same trail conditions and grades it automatically on code quality and journey outcome.

The purpose of this coursework is to consolidate your knowledge of inheritance, polymorphism, interfaces, encapsulation and composition by building a decision AI that tries to survive 2,000 miles of river crossings, storms, broken axles and hunger.

Your submission is compiled by the platform against a small contract library (`oregon-api`); this brief and the API Reference are its full definition — if a class or method is not in those documents, it does not exist for you. The engine invokes your code at well-defined decision points; it never asks you "what now?" interactively — your objects **are** the decisions. Every team faces the same three random seeds, so outcomes differ only by the quality of your code, never by luck of the draw.

A complete starter example (`sample`) is provided with this brief.

---

1. The Game You Are Playing

1.1 The journey

- Get the party from mile 0 to mile 2,000 within 120 turns.
- The run ends early only if every member dies. If all oxen die the wagon cannot move — the clock and hunger keep running.
- Each turn runs in a fixed order: party strategy → travel (needs oxen) → eat → random event → each member acts → bookkeeping → survival check.

1.2 Resources

RESOURCE	YOU START WITH*	WHAT IT DOES
Food	bought at the store	Eaten every turn: rations × living members, plus 2 per ox. Running out is dangerous — §1.3.
Oxen	bought at the store	The engine of the wagon: 4+ oxen = full pace; fewer slow you proportionally; 0 = stranded. Oxen eat 2 food each per turn and can die on grueling pace or risky choices.
Spare parts	bought at the store	Consumed when repairing breakdowns.
Ammunition	bought at the store	The entry ticket for hunting.
Money	your unspent store wallet	Pays for river ferries and fleeing bandits, and buys goods at trading posts. Selling goods at a trading post is the only way to earn more.
Health (per member)	set at construction	Damaged by events, grueling pace, starvation. 0 = dead.

* You choose your starting supplies yourself — see §1.6, the General Store.

1.3 Starvation

If the party cannot feed everyone in a turn, **every member** loses health — and the penalty grows with each consecutive hungry turn: 5, 8, 12, 17, then 23 per turn thereafter. While starving, HEAL and REST have no effect. Eating fully resets the sequence.

1.4 The six member actions

Each living member performs exactly one action per turn by returning it from `act()`. Actions are binary — a member chooses *what* to do, never how hard; every yield, cost and perk is a fixed rule the engine applies.

ACTION	EFFECT
`HUNT`	Costs 1 ammo; yields 7 food (Hunter: 10). Your main food source — the quiver is a food budget, and the wagon carries at most 500 food (surplus spoils).
`HEAL`	Self +10 health; a Doctor's HEAL instead treats the most wounded member of the party (+15). Disabled while starving.
`REST`	Self +3 health. Disabled while starving.
`SCOUT`	Finds a shortcut: +3 miles this turn (Scout: +6).
`MOVE`	Drives the wagon on: +5 miles, −1 own health — for everyone, Drivers included.
`DEFEND`	If a threatening event hit this turn, every living member recovers +3 health. The bonus applies **once per turn** however many members defend — one defender is enough.

1.5 Random events

Each turn, with probability rising from ~15% to ~43% as you progress, one of nine events strikes: river crossing, storm, breakdown, illness, attack, trading post, hunting opportunity, bad trail, theft. Your `Resolver` answers every event with one of three postures: **SAFE** (cautious — usually costs money or goods), **RISKY** (gamble — can pay off big, can hurt, and on some events can kill an ox), or **AVOID** (stay clear — usually costs miles). Event severity (1–10, rising with progress) scales the consequences. The full numeric matrix (every event × posture × severity) is published in the Engine Rules Spec — **the game is fully transparent; strategy beats dice.**

1.6 The General Store — outfitting your wagon

Before turn 1 there is a one-time shopping trip. Every team gets the same wallet: **$2,000** — exactly the price of the classic wagonload (500 food, 80 ammo, 5 oxen, 2 parts, $300 change for the road). Buy what you want, in any mix; quantities are clipped to what your wallet affords (and food to what the wagon can carry — at most 500), and whatever you don't spend stays as your starting money.

```
@Override
public List<Purchase> buySupplies() {
    return List.of(new Purchase(ResourceType.OXEN, 5),    // strength and speed
                   new Purchase(ResourceType.FOOD, 500),   // the classic load
                   new Purchase(ResourceType.AMMO, 80),
                   new Purchase(ResourceType.SPARE_PARTS, 2));
}
```

Return `null` and the engine buys the classic basket for you. Party size does not change the wallet — a small party is a rich party, a large party has more hands. Your call.

1.7 The trading post

The trading post event is a market: every good except money is bought and sold at posted prices, which move with the market's mood (severity): a bargain market prices things at ×0.7, a fair one ×1.0, a steep one ×1.4. The merchant pays only 70% of the buy price when you sell. Your resolver can read the posted prices before deciding, and performs **one transaction** per visit:

```
Resolution.buy(ResourceType.OXEN, 1);       // buy 1 ox at the posted price
Resolution.sell(ResourceType.AMMO, 30);     // sell 30 ammo for money
Resolution.haggleBuy(ResourceType.FOOD, 50); // haggle first: success ≈ ×0.7, failure = no
        deal
Resolution.decline();                       // browse and walk away
```

Base prices: food 2, ammo 3, parts 30, oxen 80. If you cannot afford a single unit, no deal happens; quantities are clipped to what you can afford (buying, at most 50 units per good per visit) or hold (selling).

---

2. What You Must Implement

2.1 The cast: story roles vs. code

IN THE STORY	IN YOUR CODE	REQUIRED?
The expedition charter — who you hired, and who holds which office	`PartyFactory` : your single entry point	Yes
The shopping trip at the general store before you leave	`PartyFactory.buySupplies()` — your shopping list	Optional (null = classic basket)
A traveller with a trade (hunter, doctor, …) doing one chore each morning	a class extending a role class; its `act(Situation)` returns an `Action`	Yes
The wagon master's standing orders — pace, rations, rest days	`PartyStrategy.decide(...)` returning a `Directive`	Optional
The spokesperson answering each obstacle: ford the river or pay the ferryman?	`Resolver.resolve(...)` returning a `Resolution`	Optional

`Situation` (the scout's morning briefing — everything the party can perceive) and `Event` (the obstacle) are provided to your code; so are `ResourceType` amounts in the shared `Inventory` via `PartyState`.

A first word about interfaces. Several of the contracts above — `PartyStrategy`, `Resolver` — are Java interfaces: a list of method signatures without bodies. An interface says *what* must be provided, never *how*. Your class implements an interface by supplying the bodies:

```java
public class CarefulResolver implements Resolver {
    @Override
    public Resolution resolve(Event event) {
        return Resolution.of(Resolution.Choice.SAFE); // always play it safe
    }
}
```

Why the engine loves them: it only ever knows the contract, never your class names — it can call `resolve(...)` on any object that implements `Resolver`. That is polymorphism at work, and it is exactly how your `Member` subclasses all answer `act()`. Three practical notes:

1. Unlike `extends` (one superclass only), a class may implement many interfaces. Use `extends` for identity ("is a Hunter"); use `implements` for capabilities ("can resolve events").
2. Interfaces may contain **default methods** — methods with bodies. `PartyFactory`'s optional hooks are default methods that return `null`, so you only override the ones you want.
3. A one-method interface can be written as a lambda: e.g. `return event -> Resolution.decline();` from your factory.

2.2 Required: the entry point

Exactly one implementation of `PartyFactory`:

```java
public interface PartyFactory {
    List<Member> createMembers();               // your party: 3–5 members
    default List<Purchase> buySupplies() { return null; }  // null = classic basket
    default PartyStrategy strategy() { return null; }      // null = engine default
    default Resolver resolver()    { return null; }        // null = engine default
}
```

`createMembers()` must return **3 to 5 members**; any other count (or null/empty) fails the run with an explicit error.

2.3 Required: members and roles

Every member extends `Member` — usually through one of the **five role classes** we ship:

```java
public abstract class Member implements Actable {
    public static final int MAX_HEALTH = 100;
    protected Member(String name, int startingHealth);  // health in [0, 100]
    public abstract Action act(Situation situation);    // ← your behaviour
    public final String getName();
    public final int getHealth();
    public final boolean isAlive();
}
```

Roles. The library provides five role classes, each an empty subclass of `Member` with two constructors — `Role(String name)` (full health) and `Role(String name, int hp)`. Each role grants one perk. Hunter, Doctor and Scout perks change what that member's **own** actions do; Mechanic and Driver perks are **auras** the engine applies while the member lives — no code needed, just keep them alive:

ROLE CLASS	PERK
`Hunter`	HUNT yields 10 food per ammo instead of 7.
`Doctor`	HEAL restores +15 base — and goes to the most wounded member of the party, not just themself.
`Mechanic`	while any Mechanic lives, safe breakdown repairs consume no spare part.
`Scout`	SCOUT finds +6 miles instead of +3.
`Driver`	while any Driver lives, grueling pace costs 2 health per member (not 4) and the ox-death chance drops to 7% (not 15%).

What writing a role extension looks like — this is where you practise inheritance and polymorphism:

```
/** Hunts when food runs low, stands firm when danger strikes, rests otherwise. */
public class TrailHunter extends Hunter {      // ← extends the ROLE, not Member
    private static final int LOW_FOOD = 30;    // private field: tune your own policy

    public TrailHunter(String name) {
        super(name);                           // chain to the role constructor
    }

    @Override
    public Action act(Situation s) {           // one action per turn
        if (s.isThreatened()) return Action.of(Action.Type.DEFEND);
        if (s.getFoodLevel() < LOW_FOOD) return Action.of(Action.Type.HUNT);
        return Action.of(Action.Type.REST);
    }
}
```

Rules of the road for member classes:

- You must override `act(Situation)`; everything else on `Member` is final — state is encapsulated — the engine applies every health change from the actions you return.
- Add your own private fields (thresholds, memory of past turns, …). Do not add public fields — the OOP score penalises them.
- You may create as many member classes as you like, and take the same role more than once — self-perk copies each improve their **own** actions (two Hunters each hunt better; they do not boost each other), and auras do not stack (one living Mechanic or Driver is enough).
- You may also subclass `Member` directly (a trade of your own invention). It compiles and runs with all actions available — but receives no perk.
- Constructors: `super(name)` for full health or `super(name, hp)` to start wounded (never above 100 — it throws).

2.4 Optional: party strategy (PartyStrategy / Directive)

```
public interface PartyStrategy { Directive decide(PartyState state); }

new Directive(Directive.Pace.GRUELING, Directive.Ration.MEAGER);    // pace, rations
```

- **Pace:** SLOW 12 / NORMAL 20 / GRUELING 28 miles per turn (at full oxen); grueling costs every member 4 health and each turn risks killing an ox (15%) — both halved while a Driver lives.
- **Rations:** BARE 1 / MEAGER 2 / FULL 5 food per member per turn (BARE slowly wears the party down: −1 HP each per turn); fully-fed FULL heals everyone +2.
- Returning `null` (or throwing) makes the engine use its default for that turn: NORMAL pace, FULL rations — note that FULL eats a lot of food.

2.5 Optional: event resolution (Resolver / Resolution / Event)

```
public interface Resolver { Resolution resolve(Event event); }
```

What you can read on an `Event`: `kind()` (e.g. `"RIVER"`, `"TRADER"`), `severity()` (1–10), `getTurn()`, `getDescription()`, and — on trading posts — `isMarket()`, `buyPrice(ResourceType)`, `sellPrice(ResourceType)`.

What you may return (`Resolution`): `of(Choice.SAFE | RISKY | AVOID)`, or the market factories `buy(good, qty)` / `sell(good, qty)` / `haggleBuy(good, qty)` / `haggleSell(good, qty)` / `decline()`. Haggle success chance is `0.5 + (severity − 5) × 0.04` — steeper markets have more room to negotiate; failure kills the deal.

One fairness rule to know: a SAFE option whose money cost you cannot afford is automatically downgraded to AVOID — the ferryman does not take IOUs.

---

3. Deliverables

Submit on the course platform (one submission per team):

Java source code: one or more `.java` files, or a single `.zip` containing them (`.java` entries only).

- Files at the archive root, no package declarations, UTF-8, one public class per file, at most 20 files.
- Exactly one `PartyFactory` implementation; 3–5 members.

No written report is required: the platform's automatic analysis replaces it. What you should do instead is iterate — submit, watch the replay, fix, resubmit.

---

4. Sandbox and Fair Play

Your code runs in an isolated, instrumented sandbox. The rules:

1. **Compilation:** JDK 21; the classpath contains only `oregon-api` — you cannot import engine or platform classes.
2. **Banned APIs — using any of these rejects the submission (score 0):** reflection (`java.lang.reflect`, `Class.forName`, `java.lang.invoke`), `Runtime` / `ProcessBuilder`, thread creation and `java.util.concurrent`, `System.exit`, `ClassLoader`, `java.util.Random`, **modifying engine-owned state directly** (health, supplies and the roster change only through the engine applying your returned actions and choices), **file access (`java.io`, `java.nio`)**, **network access (`java.net`)**, and **environment reads (`System.getenv`, `System.getProperty`)**. Your decision code

needs none of them. (Console printing with `System.out` is the one allowed `java.io` use; `Math.random()` and time APIs are permitted — the sandbox makes them deterministic.)

3. **Budgets:** each of your calls (`act`, `decide`, `resolve`, `buySupplies`) gets ≈100,000 operations before it is skipped that turn; the whole run gets a 10-second wall clock per seed.
4. **Uncaught exceptions fail the journey:** a crash caps your final mark at 40 (§5). Program defensively: check for null, validate indices, never let `act()` throw — including building an `Action` around a null type.
5. **Determinism and fairness:** all teams run the same three seeds; your marks are reproducible. Code-similarity detection runs continuously across all submissions.

---

5. Marking

All four dimensions are computed automatically from your latest submission:

```
FINAL = 0.20 × OOP + 0.50 × Game Outcome + 0.15 × Naming + 0.15 × Efficiency   (max 100)
Game Outcome = 60% distance + 25% survivors + 15% supplies (pro-rated by distance),
averaged over 3 seeds
```

The static dimensions (OOP, Naming, Efficiency) are your base points — earn them with clean code. The journey outcome is half the mark, and the trail is not gentle: most parties that coast die on it.

CRITERIA	EXCELLENT (FULL MARKS)	GOOD	SATISFACTORY	POOR / MISSING	MARKS
Object-Oriented Design	Clear class hierarchy through role classes; meaningful names; full encapsulation (no public fields, controlled mutation); genuine polymorphism (behaviour varies by subclass, no `instanceof` chains or downcasts); interfaces used where the contract offers them.	Mostly clean design; minor encapsulation or hierarchy lapses.	Some OOP applied but inconsistent; procedural habits visible.	Procedural code; public fields; no meaningful inheritance.	20
Game Outcome	Reaches Oregon on all seeds with survivors and healthy supplies; sound store outfitting and food/oxen/money management; events handled sensibly.	Completes most journeys; occasional losses from resource mismanagement.	Partial progress; recurring starvation, strandings or deaths.	Early deaths, crashes or non-running code (functional gate applies).	50
Naming & Readability	Classes PascalCase, methods/variables camelCase throughout; names reveal intent; no single-letter or vague names.	Mostly good; a few unclear names.	Frequent unclear or abbreviated names.	Single-letter / misleading names widespread.	15
Execution Efficiency	Tight decision logic; operations well within budget on all seeds.	Slightly over reference budget on some seeds.	Heavy loops in per-turn code; budget skips occur.	Pathological computation; actions regularly skipped.	15

The exact automated checklists for OOP, Naming and Efficiency are published on the platform's marking page — no hidden criteria. (Instructors may moderate marks offline after export; the platform score is the starting point.)

Functional gate: if the code does not compile, crashes, or times out, the Game dimension scores 0 and the final mark is capped at 40. Rejection (banned APIs) scores 0. Advanced extensions (multi-phase strategies, market arbitrage, richer role hierarchies) are encouraged — they feed directly into the OOP and Game dimensions.

6. Hints and FAQ

- **Start from the sample.** It is complete and runnable — and deliberately mediocre. Beating it is your first milestone.
- **The replay is your debugger.** After each submission, open the animated replay: every turn's event, choice, actions, health and supplies. When something dies, the replay tells you why.
- **Food first.** The fastest way to fail is ignoring the escalating starvation rule. HUNT early, buy at bargain markets, watch the oxen's appetite (they eat before your strategy feels comfortable).
- **The store is yours.** You have exactly $2,000; the classic load is one option, not a recommendation.
- **Situation also exposes** `getAmmoLevel()`, `getOxenCount()` and `getPartySize()` — the scout's briefing covers the whole wagon.
- `act()` returning `null` is legal (the member idles that turn) — but an idle party is a dead party.
- **Do not fight the sandbox.** Random sources are neutralised; strategy beats dice.
- **Common zero-mark traps:** `java.util.Random`; file or network access; a second `PartyFactory`; an exception escaping `act()` (including a `null` action type).
- Ask administrative/procedural questions on the LM Q&A Forum — instructors answer there for the benefit of all.